
FACE
Prep

Sorted Unique Permutation

Print distinct sorted permutations with duplicates allowed in input

Write a program to print all distinct permutations of a given string in sorted order. Note that the input string may contain duplicate characters.
In mathematics, the notion of permutation relates to the act of arranging all the members of a set into some sequence or order, or if the set is already ordered, rearranging (reordering) its elements, a process called permuting.

Examples:

Input : BAC
Output : ABC ACB BAC BCA CAB CBA
Input : AAB
Output : AAB ABA BAA
Input : DBCA
Output: ABCD ABDC ACBD ACDB ADBC ADCB BACD BADC BCAD BCDA BDAC BDCA CABD CADB CBAD CBDA CDAB CDBA DABC DACB DBAC DBCA DCAB DCBA

FACE Prep

sorted Permutations

Concept Used: The number of Strings generated by a string of distinct characters of length ‘n’ is equal to ‘n!’. Sorting any given string and generating the lexicographically next bigger string until we reach the largest lexicographically string from those characters.

Different permutations of word “geeks”
Length of string = 5
Character ‘e’ repeats 2 times.
Result = 5!/2! = 60.

Sorted Permutations

Steps:

- **Example:** Consider a string “ABCD”.
  Step 1: Sort the string.
  Step 2: Obtain the total number of permutations which can be formed from that string.
  Step 3: Print the sorted string and then loop for the number of (permutations-1) times as 1st string is already printed.
  Step 4: Find the next greater string.

```
1.  import java.util.Arrays;
2.  import java.util.HashSet;
3.  import java.util.Set;

4.  public class Permutations {
5.      public static void main(String[] args) {
6.          String input = "BAC";
7.          distinctPermutations(input);
8.      }

9.      public static void distinctPermutations(String input) {
10.         char[] chars = input.toCharArray();
11.         Arrays.sort(chars);
12.         input = new String(chars);
13.         permute(input.toCharArray(), 0);
14.     }

15.     public static void permute(char[] chars, int index) {
16.         if (index == chars.length - 1) {
17.             System.out.println(String.valueOf(chars));
18.             return;
19.         }
```

```
1.      Set<Character> used = new HashSet<>();
2.          for (int i = index; i < chars.length; i++) {
3.              if (used.contains(chars[i]))
4.                  continue;

5.              used.add(chars[i]);
6.              swap(chars, index, i);
7.              permute(chars, index + 1);
8.              swap(chars, index, i);
9.          }
10.     }

11.     public static void swap(char[] chars, int i, int j) {
12.         char temp = chars[i];
13.         chars[i] = chars[j];
14.         chars[j] = temp;
15.     }
16. }
```

THANK YOU



FACE Prep